Алгоритмы сжатия — Потоки данных (подпроекты 24–27)

Что именно передаётся на каждом шаге — без кода, простым языком

Схема 1 — Общий поток данных через все четыре семьи сжатия

Аналогия: как работает почтовая служба — сначала упаковываем посылку (сжатие), потом распаковываем (декомпрессия). Каждая семья алгоритмов — это другой способ упаковки.

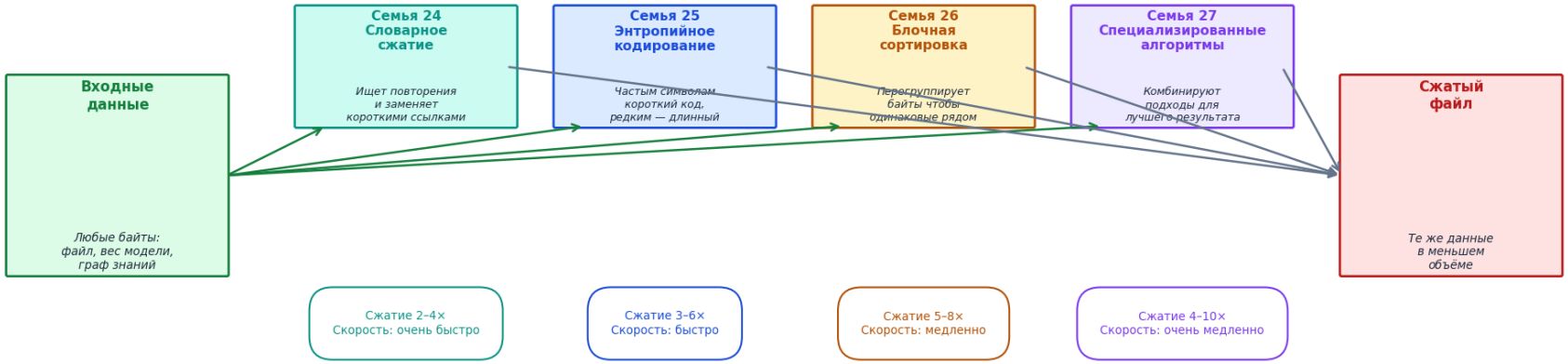


Схема 2 — Семья 24: Словарное сжатие (LZ4 стиль)

Аналогия: стенограф на совещании. Когда слышит слово которое уже записал — пишет не слово, а «смотри 5 строк назад, 3 слова». В тексте с повторениями это экономит место.

Шаг за шагом: что происходит с данными

ШАГ 1 → Исходные байты поступают в хеш-таблицу

Хеш-таблица = телефонная книга:
каждая 4-байтная последовательность → адрес где она встречалась раньше

ШАГ 2 → Для каждой позиции: было ли такое раньше?

Если ДА → запоминаем: «смотри N байт назад, скопируй M байт»
Если НЕТ → записываем байт буквально

ШАГ 3 → Буквальные байты + ссылки → выходной поток

Пример: [буква А][буква Б][ссылка: 10 назад, 5 байт][буква В]

ШАГ 4 → Сохранение: размер оригинала в заголовке

Первые 4 байта сжатого файла = длина оригинала в байтах
Нужно для точного выделения памяти при распаковке

РАСПАКОВКА → Читаем поток команд в обратном порядке

Буквальный байт → копируем в выход
Ссылка «N назад, M байт» → копируем из уже распакованного буфера

Что получается на выходе (пример: исходный код C#)

Оригинальный файл .cs	10 000 байт = 10 КБ 100%
После словарного сжатия	4 000 байт = 4 КБ 40% — экономия 60%
Плюс хранение хеш-таблицы	+ 200 байт заголовок незначительно
Скорость сжатия	~500 МБ в секунду быстрее чем читать с диска
Скорость распаковки	~2000 МБ в секунду очень быстро

Схема 3 — Семья 25: Энтропийное кодирование (Huffman)

Аналогия: азбука Морзе. Буква «Е» (самая частая в русском) — одна точка. Буква «Ъ» (самая редкая) — длинная последовательность. Huffman строит такую «азбуку» автоматически для каждого файла.

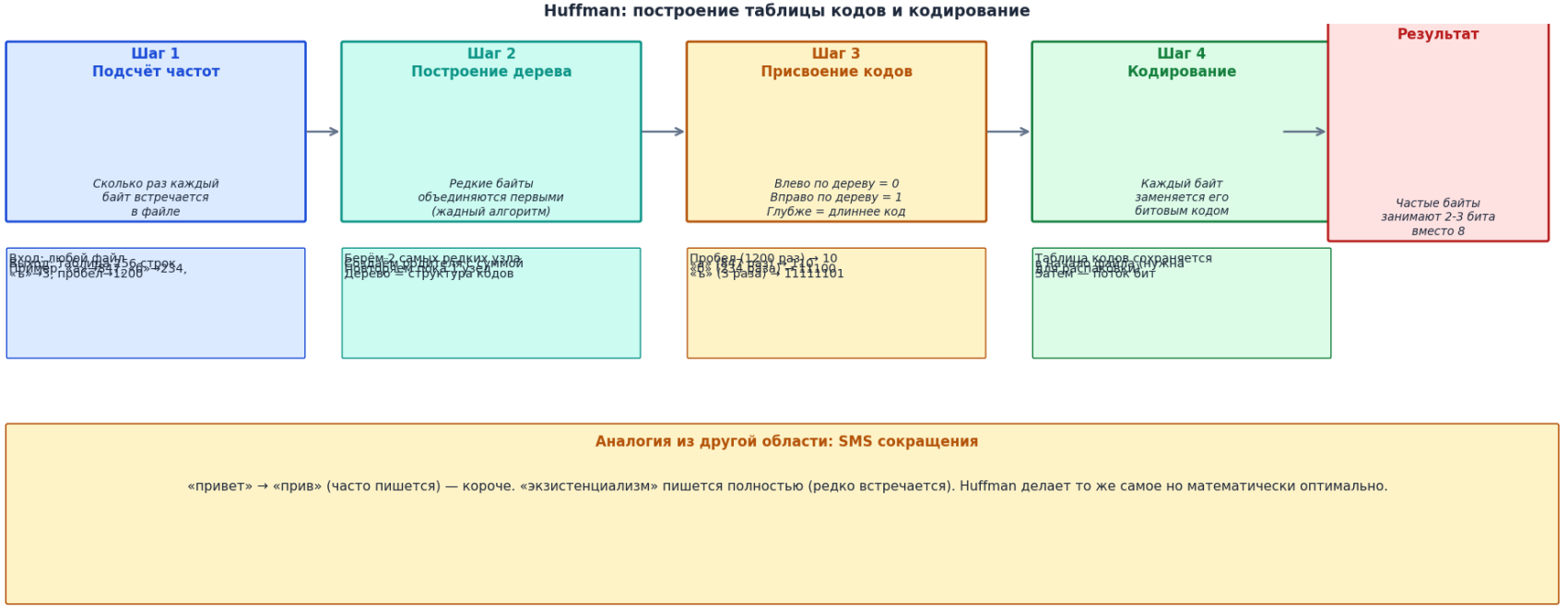
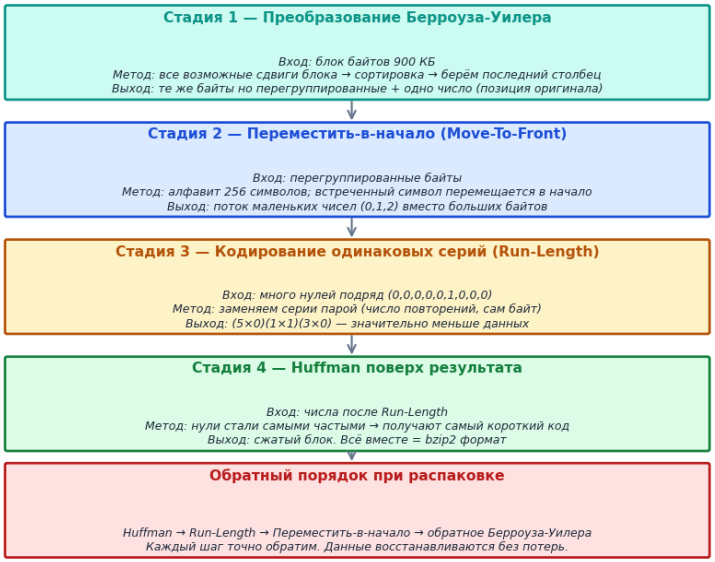


Схема 4 — Семья 26: Блочная сортировка (Преобразование Берроуза-Уилера)

Аналогия: библиотека. Берём все книги, раскладываем по алфавиту последней буквы названия. Вдруг все книги на одну тему оказались рядом — хотя мы не по теме сортировали. BWT делает то же самое с байтами.

Поток данных через три стадии



Почему группировка помогает сжатию

